UNIVERSIDAD DEL VALLE DE GUATEMALA

Facultad de Ingeniería
Departamento de Ciencias de la Computación

CC3066 — Data Science
Laboratorio 11 — MLOps: pipeline reproducible, trazabilidad y despliegue

Observatorio de Inteligencia Turística

Informe técnico de verificación y entrega

Francisco Martinez	23617
Jonathan Zacarias	231104
Mario Rocha	23501
Luis Pedro Lira	23669

Integrantes y carnés tomados de la carátula del informe del Laboratorio 10 (`reporte_final_lab10/Laboratorio_10_Documento_Diseno_Final.pdf`).

Repositorio: `https://github.com/Zacatac23/Laboratorio-_10_Dashboard.`
(debe pasar a visibilidad privada; ver §13)

Guatemala, 2 de noviembre de 2026

Resumen ejecutivo

El Observatorio publica una estimación mensual de viajeros a Guatemala (serie `Total` y siete subseries) mediante un pipeline batch reproducible: valida las fuentes con seis controles de calidad, evalúa el procedimiento sin fuga temporal frente a un baseline en los mismos meses, aplica una regla de aceptación predefinida, registra cada ejecución en MLflow y solo entonces publica de forma atómica la versión que lee el tablero.

En la verificación final (2 de noviembre de 2026): la suite de 99 pruebas automatizadas pasó (98 PASS y 1 omitida sin Docker; 99 PASS con Docker); dos ejecuciones completas en contenedor fueron idénticas (diferencia máxima 0.0 en 504 estimaciones); siete fallos controlados se detectaron sin alterar la versión aprobada; se restauró una versión anterior con verificación SHA-256; y Docker Compose funcionó desde una copia limpia con persistencia de MLflow tras `down/up`. El modelo activo (Promedio Histórico Mensual) obtiene MAE 80,365.74 y RMSE 94,175.26 viajeros/mes en 63 meses (abril 2021–junio 2026), frente a 235,730.91 y 253,203.39 del baseline Seasonal Naive en los mismos meses.

Se encontraron y corrigieron dos defectos (filtración de la URI de MLflow entre corridas y `--allowed-hosts` con puerto fijo) y un problema visual del tablero. Quedan abiertos: cotejar la matriz con el enunciado oficial (no disponible en el repositorio), hacer privado el repositorio y re-verificar desde un commit que incluya los cambios de esta etapa. Un backtest complementario muestra que, a 12 meses, el baseline supera al modelo activo en 6 de 8 cortes; se documenta como riesgo sin modificar reglas a posteriori.

1. Alcance

Tabla 1: Alcance de la estimación (fuente: `config/pipeline_config.yaml`).

Variable	`viajeros`: ingreso mensual a Guatemala; `Total` y subseries `Via_Aérea`, `Via_Terrestre`, `Via_Marítima`, `Tipo_Turista`, `Tipo_Excursionista`, `Tipo_Cruceristas`, `Tipo_Viajero`
Unidad / agregación	viajeros/mes, no negativa; mensual (`YYYY-MM-01`)
Fuentes	`series_mensuales.csv` (serie, fecha, viajeros) para modelado; `pais_mensual.csv` y `via_frontera.csv` como contexto (QC no bloqueante y monitoreo)
Periodos	Ajuste 2009-01 a 2021-03 (147 meses); evaluación 2021-04 a 2026-06 (63 meses), sin exclusiones
Modelo	Promedio Histórico Mensual (activo); Seasonal Naive lag 12 (baseline); Holt-Winters aditivo (referencia)
Factor de reparto	Proporción de cada subserie sobre el Total en el periodo de ajuste (p. ej. `Tipo_Turista` 0.7408, `Via_Terrestre` 0.5904, `Via_Aérea` 0.3759)
Salida	`data/estimaciones_aprobadas.csv` + manifiesto JSON (SHA-256, `run_id`, métricas, periodo)

2. Diagrama del pipeline

La Figura 1 resume el flujo implementado en `src/pipeline.py`: dos compuertas (calidad y aceptación), verificación de trazabilidad antes de cualquier publicación y estados finales explícitos con su código de salida.

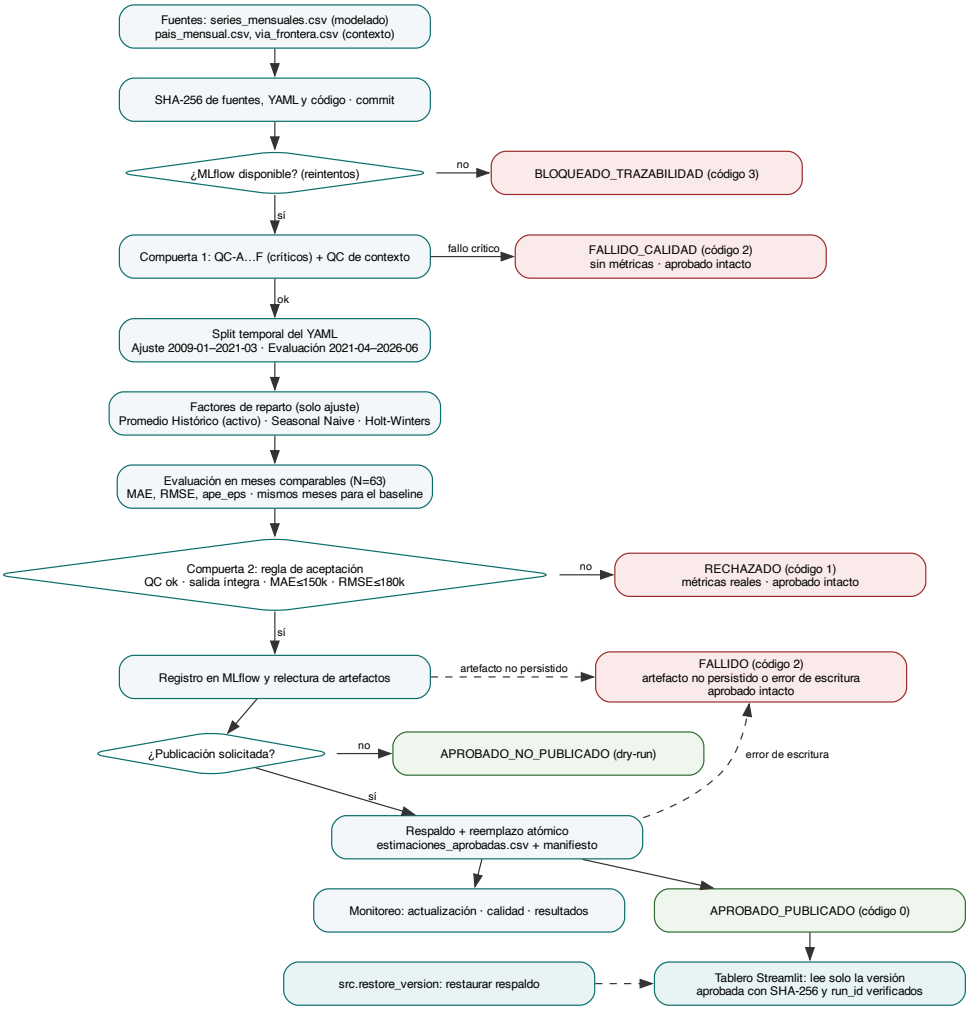


Figura 1: Pipeline real con puntos de validación, evaluación, aprobación, publicación y consulta. Fuente editable: `reporte_final_lab11/diagrama_pipeline.dot`.

3. Preparación, ejecución reproducible y dependencias

- Ejecución: `python pipeline.py --config config/pipeline_config.yaml --publish` o `--no-publish` (exactamente una). Códigos: 0 aprobado, 1 rechazado, 2 fallo, 3 bloqueado.
- Configuración única en YAML: rutas, periodos, exclusiones, modelo activo, umbrales, semilla 42 y estrategia MLflow. Variables de entorno documentadas en `.env.example`.
- Dependencias: `requirements.txt` (directas exactas) y `requirements.lock`; imagen `python:3.12.10-slim`. Entorno local verificado con Python 3.12.13 y `pip check` sin conflictos.

4. Controles de calidad

Tabla 2: Controles de `src/quality_checks.py` y resultado en las ejecuciones verificadas.

Control	Detiene si...	Resultado observado
QC-A Columnas y tipos	falta `serie`/`fecha`/`viajeros` o tipo incompatible	PASSED con datos reales; FAILED al eliminar `viajeros`
QC-B Fechas y cobertura	fecha inválida, día $\neq 1$, hueco o ventana incompleta	PASSED (210 meses continuos)
QC-C Unicidad	clave (`serie`, `fecha`) duplicada	PASSED; FAILED al duplicar una fila
QC-D Faltantes	nulos en columnas indispensables	PASSED
QC-E Valores	NaN, infinito o negativo (cero válido)	PASSED; FAILED con Total 2015-03 $= -500$
QC-F Regla metodológica	Total difiere >2 % de la suma de vías o faltan insumos	PASSED; FAILED/NO_VERIFICABLE en los fallos inyectados
Contexto	duplicados/valores en `pais_mensual`, `via_frontera`	Advertencia, no bloqueo (prueba `FuentesMultiples`)

Observación. En el pipeline QC-E se invoca sin columnas denominador; un Total de entrenamiento nulo se detecta en el cálculo de factores (estado `FALLIDO`, etapa de modelado). La contención funciona (la versión aprobada no cambia), pero el control no queda registrado como QC-E.

5. Trazabilidad con MLflow

Cada run registra: estado, commit (en contenedor, declarado por `GIT_COMMIT`), SHA-256 de las tres fuentes, del YAML y del código, semilla, split, factores, perfil del modelo, reporte QC, regla y decisión, métricas del modelo y de los baselines, monitoreo, auditoría, estimaciones candidatas y manifiesto (11 artefactos, Figura 2). Los artefactos se releen antes de publicar.

Tabla 3: Dos ejecuciones completas en el servidor MLflow de Docker Compose (`reports/reproducibilidad_lab11.json`).

Run del pipeline	ID MLflow	Estado
`run_20261102_001410_ce7eea`	`7787c39f59d64cf69394bde8de3ea709`	aprobado, no publicado (dry-run)
`run_20261102_001413_338a6e`	`575d18e094f245f1bf51697e3cd11381`	aprobado, no publicado (dry-run)

Comparación: 8 series $\times$ 63 meses con `np.allclose(atol=1e-6, rtol=0)`; diferencia máxima 0.0; `config.sha256` (`d12e42c6...`), `code.sha256` (`54c04097...`) y hashes de fuentes idénticos; 25 métricas idénticas. Estado: REPRODUCIBLE. Volátiles ignorados: `run_id`, tiempos, duración y columnas `run_id`/`fecha_generacion` del CSV candidato.

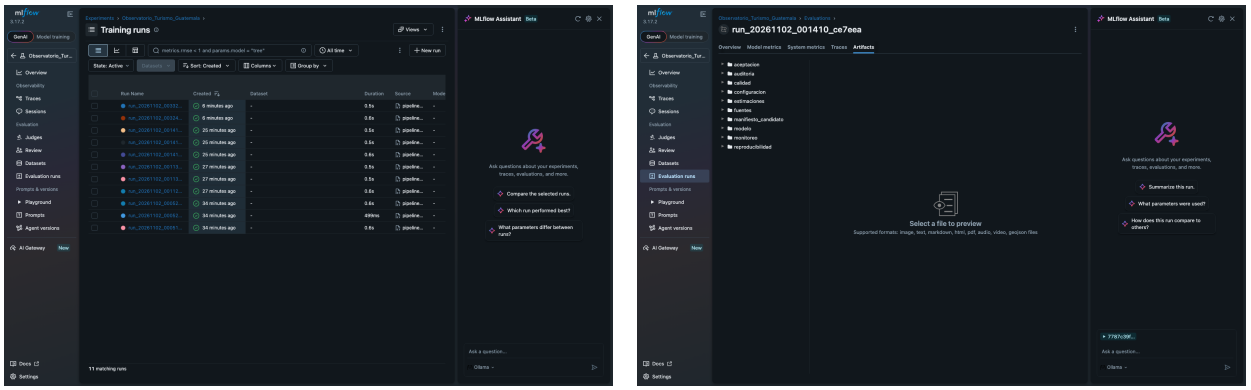


Figura 2: UI real de MLflow en el servidor de Compose (puerto 5001 por conflicto con AirPlay en macOS). Izquierda: 9 runs del experimento `Observatorio_Turismo_Guatemala`. Derecha: carpetas de artefactos del run `run_20261102_001410_ce7eea`.

Holt-Winters no es reproducible entre sistemas operativos (MAE 302,827.47 en Linux/Docker frente a 178,765.59 registrado en Windows en la etapa previa), por el optimizador numérico; no es el modelo activo. La exportación completa de runs está en `evidencias/lab11/mlflow/export_runs.json`.

6. Validación temporal

Backtest de origen fijo: ajuste hasta 2021-03 y estimación de los 63 meses siguientes sin reajuste. Solo se evalúan meses con observación finita; enero y febrero de 2026 tienen dato y se evalúan (la memoria anterior decía lo contrario). La prueba `SinFugaTemporal` demuestra que multiplicar por 10 los meses evaluados no cambia ningún parámetro, factor ni predicción. La métrica relativa es $\texttt{ape_eps} = \overline{|e|/(|y|+1)} \times 100$, no el MAPE convencional.

Tabla 4: Métricas en los mismos 63 meses (run `run_20261102_001410_ce7eea`).

Procedimiento	MAE	RMSE	ape_eps %	N
Promedio Histórico Mensual (activo)	80,365.74	94,175.26	38.67	63
Seasonal Naive lag 12 (baseline)	235,730.91	253,203.39	83.86	63
Holt-Winters aditivo (referencia)	302,827.47	327,720.16	103.99	63

Tabla 5: Backtest complementario de cortes múltiples (horizonte 12 meses, mismos meses por corte). Fuente: `evidencias/lab11/evaluacion/backtest_cortes_multiples.json`.

Origen (evalúa los 12 meses siguientes)	Activo	Seasonal Naive	Holt-Winters
2016-12	113,813.60	42,363.00	29,482.86
2017-12	129,075.01	32,755.12	25,021.62
2018-12	152,456.80	42,261.10	22,247.41
2019-12 (evalúa 2020, COVID-19)	211,364.21	306,100.68	328,357.30
2022-12	36,019.73	112,255.28	209,448.82
2023-12	58,299.74	36,983.58	21,869.34
2024-12	54,936.17	18,024.33	10,777.61
2025-06	48,607.14	12,237.42	14,590.48
MAE promedio	100,571.55	75,372.56	82,724.43

A 12 meses el baseline supera al modelo activo en 6 de 8 cortes. El promedio histórico es robusto a la ruptura de 2020–2021 (gana en 2019-12 y 2022-12), pero desaprovecha la información reciente.

No se modificó el modelo ni la regla después de observarlo; se recomienda evaluar el cambio de modelo en una versión futura con reglas fijadas de antemano.

Limitaciones de enero–febrero de 2026. 2026 es parcial (hasta junio). La comparabilidad institucional de esos meses no está certificada; si la institución la declarara no comparable, debe registrarse una exclusión en el YAML con su motivo antes de evaluar.

7. Regla de aceptación

Predefinida en el YAML y aplicada por `evaluate_acceptance_rule`: (1) todos los QC críticos aprobados; (2) salida íntegra: todas las series, sin nulos, finita y no negativa; (3) MAE $\leq$ 150,000 y (4) RMSE $\leq$ 180,000 viajeros/mes. Superar al baseline se registra (`acceptance.beats_baseline`) pero no es obligatorio. Justificación de escala: el Total observado en el periodo evaluado promedia 276,685 viajeros/mes (mediana 279,790; rango 74,914–526,190); el umbral acota errores de alrededor de la mitad del nivel típico y rechazaría a Holt-Winters (302,827). El candidato activo resultó **aprobado**. El rechazo por desempeño se verifica con datos sintéticos (`test_corrida_rechazada_registra_metricas_reales_sin_publicar`): queda `RECHAZADO`, con métricas reales y sin publicar.

8. Integración con el tablero

El tablero lee exclusivamente `estimaciones_aprobadas.csv` y su manifiesto (`data_logic.cargar_version_aprobada`); las métricas solo se muestran si SHA-256 y `run_id` coinciden. No hay caché: una publicación o restauración se ve al recargar.

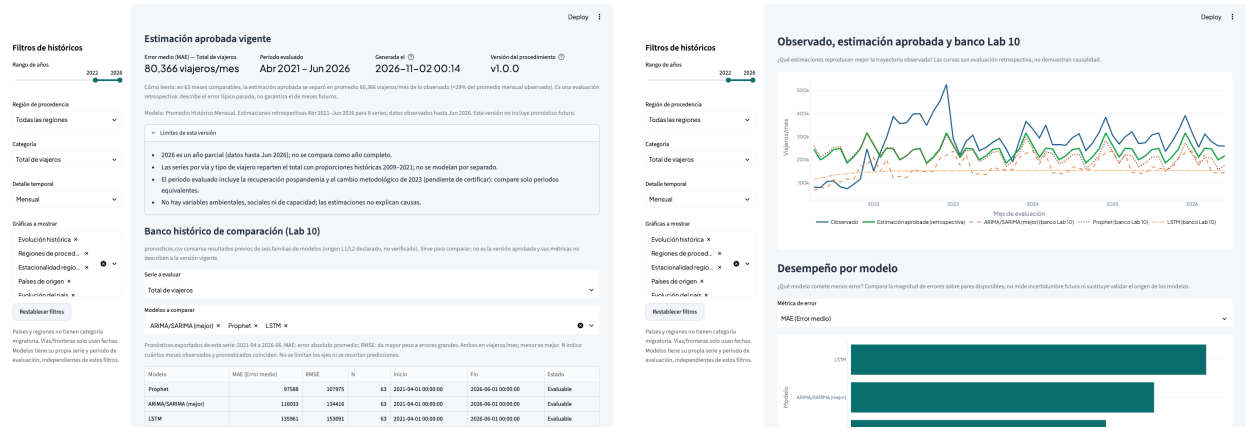


Figura 3: Tablero en Docker con la versión `run_20261102_001416_15b4a2`. Izquierda: MAE 80,366, periodo Abr 2021–Jun 2026, generada 2026-11-02 00:14, v1.0.0 y límites (2026 parcial, sin causalidad). Derecha: observado (azul), estimación aprobada (verde) y banco Lab 10 (discontinuas).

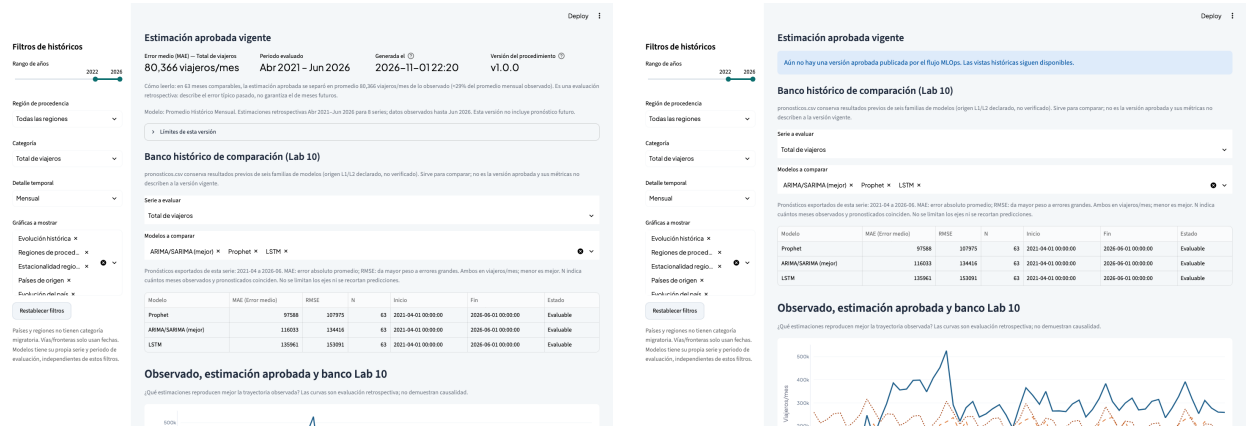


Figura 4: Izquierda: tras restaurar el respaldo, el tablero muestra la versión `run_20261101_222030_d65c3b` (generada 2026-11-01 22:20). Derecha: sin archivo aprobado aparece un aviso informativo y las vistas históricas siguen disponibles.

Cambiar filtros (selector de vista, restablecer, cambio de pestañas) no creó runs (24 antes y después) ni archivos de salida (31). Captura móvil y metadatos de cada imagen: `evidencias/lab11/dashboard/`. Se corrigió el truncado de los valores de las tarjetas (`styles.css`).

9. Monitoreo

Tabla 6: Demostración de las tres clases de monitoreo. Fuente: `evidencias/lab11/recuperacion/fallos_controlados.json`.

Clase	Situación inducida (copia temporal)	Alerta real
Línea base	Datos vigentes	0 alertas
Actualización	Serie truncada a 2026-04 (cobertura esperada 2026-06)	ALERTA: última observación 2026-04-01 no alcanza 2026-06
Calidad	25 filas duplicadas frente a la ejecución previa	ALERTA: tasa de duplicados 0.0147 > 0.0
Resultados	Total 2024-07 multiplicado por 3	REVISIÓN: variación intermensual +236.8 % y −66.0 % (>50 %)

El umbral de 50 % se justifica en el YAML: en 2009–2019 el percentil 95 de la variación intermensual del Total fue 32.4 %. Una alerta de variación pide revisión; no implica pérdida de precisión.

10. Prueba controlada de falla, recuperación y restauración

Sobre una copia temporal de `data/` (versión aprobada inicial A: `run_20261101_222030_d65c3b`, SHA-256 `d5795b8c...`) se inyectó cada falla por separado:

Tabla 7: Fallos inyectados y contención (ejecuciones registradas en el experimento `..._pruebas_controladas`).

Falla	Estado	Detección	Aprobado intacto
Columna `viajeros` ausente	FALLIDO_CALIDAD	QC-A, QC-F	Sí
Clave duplicada	FALLIDO_CALIDAD	QC-C, QC-F	Sí
Conteo negativo	FALLIDO_CALIDAD	QC-E, QC-F	Sí
Denominador cero (ajuste en 0)	FALLIDO	factores de reparto	Sí
Error de escritura (`os.replace`)	FALLIDO	publicación atómica; sin temporales	Sí
MLflow inaccesible	BLOQUEADO_TRAZ.	verificación previa	Sí
Artefacto no persistido	FALLIDO	registro verificado	Sí

Corregidos los datos, la recuperación publicó la versión B (`run_20261102_001003_405131`, `594b7cff...`). La restauración del respaldo de A con `src.restore_version` devolvió exactamente `d5795b8c...` y su manifiesto, y el tablero leyó A con metadatos verificados. En Docker se repitió la restauración con el CLI (`restauracion_docker.log`): el hash pasó de `f201419b...` a `d5795b8c...`, con respaldo de seguridad `_prerestauracion` y registro en `logs/restore_audit.jsonl`.

Defecto encontrado. En la primera corrida de esta prueba, «MLflow inaccesible» terminó `APROBADO_PUBLICADO`: `mlflow.set_tracking_uri()` escribe `MLFLOW_TRACKING_URI` en el entorno y esa variable tiene prioridad sobre el YAML, de modo que una segunda corrida en el mismo proceso ignoraba su configuración. Se corrigió en `TourismMLOpsPipeline.run()` y se agregó una prueba de regresión que falla sin la corrección.

11. Docker

Servicios: `mlflow` (SQLite y artefactos en el volumen `observatorio_turismo_mlflow_store`, `--serve-artifacts`, healthcheck), `dashboard` (monta `data/` en solo lectura) y `pipeline` (perfil `tools`). Imagen única `observatorio-turismo:lab11` (1.61 GB), usuario sin privilegios (uid 1000), puertos solo en 127.0.0.1.

Verificación desde copia limpia (`scripts/lab11/docker_verificacion.sh`, 28 pasos con código 0): inspección de la imagen sin CSV, `.db`, `.env`, claves ni `.git`; `up -d --wait`; HTTP 200 en tablero y MLflow; dos dry-run y una publicación en contenedor; `down` y `up` sin `-v` con los mismos 9 runs. La prueba automatizada `DockerCompose` también pasó. Se corrigió `--allowed-hosts`, que fijaba el puerto 5000 e impedía usar la UI con otro puerto. **Advertencia:** `docker compose down -v` borra el almacén de MLflow.

12. Pruebas automatizadas

Tabla 8: Resultados de `python -B -m unittest discover -s tests -v`. La prueba omitida sin Docker es la de Compose, que requiere `RUN_DOCKER_TESTS=1`. Fuente: `evidencias/lab11/tests_unitarios*.txt`.

Corrida	Pruebas	PASS	FAIL	SKIP
Sin Docker	99	98	0	1
Con Docker (`MLFLOW_PORT=5001`)	99	99	0	0

13. Limitaciones, riesgos y pendientes

- **Enunciado oficial no disponible:** la matriz de `docs/auditoria_final_lab11.md` sigue la numeración de la memoria; debe cotejarse con `Laboratorio_11_MLOps_2026.pdf`.

- **Commit:** las corridas usan el commit base `4d8f5ba` declarado más cambios sin commitear (identificados por `code.sha256`); tras commitear debe repetirse la verificación Docker sin `--snapshot`.
- **Privacidad:** los repositorios remotos son públicos y los CSV institucionales están versionados; el cambio de visibilidad corresponde al dueño.
- **Metodología:** a 12 meses el baseline supera al modelo activo en la mayoría de cortes; el umbral de MAE es tolerante (54 % del promedio observado); el reparto proporcional fija la estructura 2009–2021; el cambio metodológico de 2023 no está certificado.
- **Operación:** restauraciones auditadas en local, no en MLflow; Holt-Winters dependiente del entorno.

14. Conclusiones

El flujo cumple su propósito de gobernanza: ninguna de las siete fallas probadas alteró la versión aprobada ni produjo una salida sin trazabilidad, las ejecuciones son reproducibles bit a bit dentro del mismo entorno, MLflow conserva los registros tras recrear contenedores y el tablero comunica la versión vigente con su error, su fecha y sus límites. La principal mejora pendiente es metodológica (revisar el modelo activo con reglas fijadas de antemano) y administrativa (repositorio privado y cotejo con el enunciado).

Referencias a evidencias

Todas las rutas son relativas al repositorio: `docs/guia_evidencias_lab11.md` (índice), `evidencias/lab11/` (tests, Docker, MLflow, evaluación, recuperación y tablero), `reports/reproducibilidad_lab11.json`, `docs/auditoria_final_lab11.md` y `docs/resumen_entrega_lab11.md`.